



JOIN Dive Deep

젠슨

조인

발표에 앞서 이런 분들이 들으면 좋아요

- 조인 잘못 쓰면 느려진다는데...? 이야기를 들어보신 분
- MySQL의 JOIN 내부 동작에 대해 궁금하신 분

이런 내용은 다루지 않아요

- 옵티마이저의 cost 정산 방법
- Index Dive

이런 내용을 알고 있으면 좋아요

- 쿼리 실행 계획 읽는 법
- 기본적인 SQL

조인

오늘 다룰 문제 상황

orders

order_id	order_status
----------	--------------

order_item

order_item_id	order_id
---------------	----------

조인

오늘 다룰 문제 상황

```
SELECT COUNT(*)  
FROM orders o  
JOIN order_item oi ON oi.order_id = o.order_id  
WHERE o.order_status = 'DELIVERED'  
      AND oi.order_item_id BETWEEN 1 AND 600000;
```

order_status	total_count	ratio_pct
DELIVERED	3,000,000	60%
CANCELLED	500,000	10%
PAID	500,000	10%
SHIPPED	500,000	10%
ORDERED	500,000	10%

조인

MySQL의 기본 조인 동작

orders

order_id	order_status
1	DELIVERED
2	DELIVERED
3	DELIVERED
4	CANCELLED

Driving Table

order_item

order_item_id	order_id
1	1
2	1
3	1
4	2
5	2
6	3
7	3
8	10

NLJ (Nested Loop Join)

Driven Table

조인

MySQL의 기본 조인 동작

order_item

order_item_id	order_id
1	1
2	1
3	1
4	2
5	2
6	3
7	3
8	10

orders

order_id	order_status
1	DELIVERED
2	DELIVERED
3	DELIVERED
4	CANCELLED

8번의 외부 루프

조인

MySQL의 기본 조인 동작

orders가 Driving Table일 경우, 총 4회 반복

orders

order_id	order_status
1	DELIVERED
2	DELIVERED
3	DELIVERED
4	CANCELLED

order_item

order_item_id	order_id
1	1
2	1
3	1
4	2
5	2
6	3
7	3
8	10

조인

MySQL의 기본 조인 동작

order_item이 Driving Table일 경우, 총 8회 반복

order_item

order_item_id	order_id
1	1
2	1
3	1
4	2
5	2
6	3
7	3
8	10

orders

order_id	order_status
1	DELIVERED
2	DELIVERED
3	DELIVERED
4	CANCELLED

같은 양을 읽어도, 외부 루프의 횟수가
차이난다!

조인

MySQL의 기본 조인 동작

FK에 인덱스가
안 걸려있다면?

조인

MySQL의 기본 조인 동작

외부 루프 횟수만큼 풀테이블 스캔이 발생한다!!!

orders

order_id	order_status
1	DELIVERED
2	DELIVERED
3	DELIVERED
4	CANCELLED

order_item

order_item_id	order_id
1	3
2	1
3	10
4	9
5	1
6	1
7	7
8	8

조인

Hash JOIN

Hash Join

조인

orders

order_id	order_status
1	DELIVERED
2	DELIVERED
3	DELIVERED
4	CANCELLED

풀테이블 스캔

해시 테이블

order_item

order_item_id	order_id
1	1
2	1
3	1
4	2
5	2
6	3
7	3
8	10

Key	value
1	(1, DELIVERED)
2	(2, DELIVERED)
3	(3, DELIVERED)
4	(4, CANCELLED)

풀테이블 스캔 1회만으로도, 상수 시간에 접근 가능

조인

order_item

order_item_id	order_id
1	1
2	1
3	1
4	2
5	2
6	3
7	3
8	10

풀테이블 스캔

해시 테이블

Key (order_id)	value
1	(1, 2, 3)
2	(4, 5)
3	(6, 7)
10	(8)

orders

order_id	order_status
1	DELIVERED
2	DELIVERED
3	DELIVERED
4	CANCELLED

조인

Hash JOIN

orders 해시 테이블

Key	value
1	(1, DELIVERED)
2	(2, DELIVERED)
3	(3, DELIVERED)
4	(4, CANCELLED)

저장된 행 수: 4

order_item 해시 테이블

Key (order_id)	value
1	(1, 2, 3)
2	(4, 5)
3	(6, 7)
10	(8)

저장된 행 수: 8

해시 테이블은 메모리에 저장된다

하지만 join buffer size 초과 시 디스크에 저장

조인

오늘 다룰 문제 상황

orders

order_id	order_status
----------	--------------

order_item

order_item_id	order_id
---------------	----------

조인

오늘 다룰 문제 상황

```
SELECT COUNT(*)  
FROM orders o  
JOIN order_item oi ON oi.order_id = o.order_id  
WHERE o.order_status = 'DELIVERED'  
      AND oi.order_item_id BETWEEN 1 AND 600000;
```

order_status = 'DELIVERED' : 300만 건

order_item: 60만 건

order_status	total_count	ratio_pct
DELIVERED	3,000,000	60%
CANCELLED	500,000	10%
PAID	500,000	10%
SHIPPED	500,000	10%
ORDERED	500,000	10%

	Order_item	orders
외부 루프	60만 * 1행	300만 * 1.83행
가져오는 행	60만 행	약 549만 행
결과	359,351행	359,351행

조인

예시 상황

???? orders를 Driving으로 선택

```
-> Aggregate: count(0)
(cost=1.05e+6 rows=1) (actual time=3899..3899 rows=1 loops=1)
-> Nested loop inner join
(cost=1.04e+6 rows=128505) (actual time=2.22..3890 rows=359351 loops=1)
-> Filter: (o.order_status = 'DELIVERED')
(cost=503246 rows=485758) (actual time=2.06..940 rows=3e+6 loops=1)
-> Table scan on o
(cost=503246 rows=4.86e+6) (actual time=2.04..651 rows=5e+6 loops=1)
-> Filter: (oi.order_item_id between 1 and 600000)
(cost=0.917 rows=0.265) (actual time=901e-6..918e-6 rows=0.12 loops=3e+6)
-> Covering index lookup on oi using idx_order_item_order_id (order_id=o.order_id)
(cost=0.917 rows=1.78) (actual time=627e-6..806e-6 rows=1.83 loops=3e+6)
```

조인

예시 상황

왜??? 바본가??

옵티마이저는 데이터 분포도를 모른다...!!

조인

예시 상황

```
SELECT COUNT(*)  
FROM orders o  
JOIN order_item oi ON oi.order_id = o.order_id  
WHERE o.order_status = 'DELIVERED'  
AND oi.order_item_id BETWEEN 1 AND 600000;
```

order_status	total_count	ratio_pct
DELIVERED	3,000,000	60%
CANCELLED	500,000	10%
PAID	500,000	10%
SHIPPED	500,000	10%
ORDERED	500,000	10%

인덱스가 걸려있지 않으면,
데이터 분포를 모름

동등조건 '=' : 약 10%
Between, Like : 약 11%
<, <=, >, >= : 약 33%

...

조인

예시 상황

```
SELECT COUNT(*)  
FROM orders o  
JOIN order_item oi ON oi.order_id = o.order_id  
WHERE o.order_status = 'DELIVERED'  
AND oi.order_item_id BETWEEN 1 AND 600000;
```

order_status	total_count	ratio_pct
DELIVERED	3,000,000	60%
CANCELLED	500,000	10%
PAID	500,000	10%
SHIPPED	500,000	10%
ORDERED	500,000	10%

실제 데이터: 3,000,000
추정 데이터: 500,000

500,000 < 600,000 이므로,
orders를 Driving Table로 설정!

조인

예시 상황

해결 방안

- 1) 인덱스 걸기
- 2) 히스토그램 생성하기

조인

히스토그램

ANALYZE TABLE orders UPDATE HISTOGRAM ON order_status;

order_status	total_count	ratio_pct
DELIVERED	3,000,000	60%
CANCELLED	500,000	10%
PAID	500,000	10%
SHIPPED	500,000	10%
ORDERED	500,000	10%

value	cumulative_freq
CANCELLED	0.09862858130377355
DELIVERED	0.7006085593314488
ORDERED	0.8002246608485267
PAID	0.9005937465282492
SHIPPED	1.0

조인

```
-> Aggregate: count(0)
(cost=3.75e+6 rows=1) (actual time=3771..3771 rows=1 loops=1)
-> Nested loop inner join
(cost=3.68e+6 rows=773576) (actual time=0.28..3763 rows=359351 loops=1)
-> Filter: (o.order_status = 'DELIVERED')
(cost=502433 rows=2.92e+6) (actual time=0.264..880 rows=3e+6 loops=1)
-> Table scan on o
(cost=502433 rows=4.86e+6) (actual time=0.262..600 rows=5e+6 loops=1)
-> Filter: (oi.order_item_id between 1 and 600000)
(cost=0.907 rows=0.265) (actual time=881e-6..897e-6 rows=0.12 loops=3e+6)
-> Covering index lookup on oi using idx_order_item_order_id (order_id=o.order_id)
(cost=0.907 rows=1.78) (actual time=612e-6..787e-6 rows=1.83 loops=3e+6)
```

3771ms -> 409 ms
약 9.2배 차이!

```
-> Aggregate: count(0)
(cost=1.6e+6 rows=1) (actual time=409..409 rows=1 loops=1)
-> Nested loop inner join
(cost=1.52e+6 rows=747306) (actual time=1.15..401 rows=359351 loops=1)
-> Filter: (oi.order_item_id between 1 and 600000)
(cost=250088 rows=1.24e+6) (actual time=1.13..85.3 rows=600000 loops=1)
-> Index range scan on oi using PRIMARY over (1 <= order_item_id <= 600000)
(cost=250088 rows=1.24e+6) (actual time=1.13..63.3 rows=600000 loops=1)
-> Filter: (o.order_status = 'DELIVERED')
(cost=0.926 rows=0.602) (actual time=424e-6..454e-6 rows=0.599 loops=600000)
-> Single-row index lookup on o using PRIMARY (order_id=oi.order_id)
(cost=0.926 rows=1) (actual time=314e-6..331e-6 rows=1 loops=600000)
```

조인

마무리

해결 방법은 매우 간단하지만,
JOIN 내부 동작을 이해하고 있어야 한다

